Contents

1 Basic 1
1.1 default . . . 1
1.2 奇妙的東西 . . . 1
1.3 random . . . 1

2 Math 1
2.1 basic . . . 1
2.2 快速冪 . . . 1
2.3 大數運算 . . . 1
2.4 極座標排序 . . . 2
2.5 ExGCD . . . 2
2.6 Phi . . . 2

3 Binary Search 2
3.1 二分搜 . . . 2
3.2 三分搜 . . . 3

4 DataStructure 3
4.1 DSU(並查集) . . . 3
4.2 FenwickTree/BIT . . . 3
4.3 二維 BIT . . . 3
4.4 Seg/線段樹 . . . 3

5 Graph 4
5.1 Topo/拓樸 . . . 4
5.2 bellman/邊長鬆弛 . . . 4
5.3 ola/歐拉 . . . 4
5.4 LCA/最近公共祖先 . . . 4
5.5 ConvexHull/凸包 . . . 5
5.6 km/二分圖最大權匹配 . . . 5

6 dynamic programming 5
6.1 SOS DP . . . 5

7 Flow 6
7.1 Dinic . . . 6

8 String 6
8.1 SAIS . . . 6

9 酷東西可能會用到 6
9.1 areaofrect/矩形聯集面積 . . . 6

10 Math2 7
10.1 Theorem . . . 7
10.2 Prime . . . 7

1 Basic

1.1 default

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long
#define idonthavegirlfriend ios::sync_with_stdio(0),cin
    .tie(0);
#define pb push_back
#define endl "\n"
#define all(v) (v).begin(),(v).end()
void solve(){

}
signed main(){
    idonthavegirlfriend
    int _=1;
    //cin>>_;
    while(_--){
        solve();
    }
}
```

1.2 奇妙的東西

```cpp

priority_queue<T> //由大到小
priority_queue<T,vector<T>,greater<T>> //由小到大
__builtin_popcountll// 二進位有幾個1(記得這是long long)
__builtin_clzll         // 左起第一個1之前0的個數
__builtin_parityll      // 1的個數的奇偶性
__builtin_mul_overflow(a,b,&h) // a*b是否溢位,h = a * b
    ;
__builtin_add_overflow(a,b,&h)
```

1.3 random

```cpp
mt19937 mt(chrono::steady_clock::now().time_since_epoch
    ().count());
//mt19937_64 mt() -> return randnum
int randint(int l, int r){
    uniform_int_distribution<> dis(l, r); return dis(mt
        );
}
```

2 Math

2.1 basic

```
[a,b] 有包含
(a,b) 沒包含
```

2.2 快速冪

```cpp
int ksm(int x, int y) {
    int ans = 1;
    while (y) {
        if (y & 1)
            ans = ans * x;
        x = x * x;
        y >>= 1;
    }
    return ans;
}
```

```cpp
// 矩陣乘法
vector<vector<int>> mul(const vector<vector<int>> &A,
                        const vector<vector<int>> &B) {
    int n = A.size();
    vector<vector<int>> C(n, vector<int>(n, 0));

    for (int i = 0; i < n; i++) {
        for (int k = 0; k < n; k++) if (A[i][k]) {
            for (int j = 0; j < n; j++) {
                C[i][j] = (C[i][j] + A[i][k] * B[k][j])
                    % MOD;
            }
        }
    }
    return C;
}
```

```cpp
// 矩陣快速冪
vector<vector<int>> ksm(vector<vector<int>> base, int
    exp) {
    int n = base.size();
    vector<vector<int>> res(n, vector<int>(n, 0));

    for (int i = 0; i < n; i++) res[i][i] = 1;

    while (exp > 0) {
        if (exp & 1) res = mul(res, base);
        base = mul(base, base);
        exp >>= 1;
    }
    return res;
}
```

2.3 大數運算

```cpp
//加法
string add(string num1, string num2) {
    string res = "";
    int i = num1.length() - 1, j = num2.length() - 1,
        carry = 0;
    while (i >= 0 || j >= 0 || carry) {
        int sum = carry + (i >= 0 ? num1[i--] - '0' :
            0) + (j >= 0 ? num2[j--] - '0' : 0);
        carry = sum / 10;
        res += to_string(sum % 10);
    }
    reverse(res.begin(), res.end());
    return res;
}

//減法
string sub(string num1, string num2) {
```

```
    string res = "";
    int i = num1.length() - 1, j = num2.length() - 1,
        borrow = 0;
    while (i >= 0) {
        int sub = (num1[i--] - '0') - (j >= 0 ? num2[j
            --] - '0' : 0) - borrow;
        if (sub < 0) {
            sub += 10;
            borrow = 1;
        } else {
            borrow = 0;
        }
        res += to_string(sub);
    }
    while (res.length() > 1 && res.back() == '0') res.
        pop_back(); // 去除前導0
    reverse(res.begin(), res.end());
    return res;
}

//乘法
string mul(string num1, string num2) {
    int n = num1.size(), m = num2.size();
    vector<int> res(n + m, 0);

    for (int i = n - 1; i >= 0; i--) {
        for (int j = m - 1; j >= 0; j--) {
            int p = (num1[i] - '0') * (num2[j] - '0');
            int sum = p + res[i + j + 1];

            res[i + j + 1] = sum % 10;
            res[i + j] += sum / 10;
        }
    }
    string ans = "";
    for (int x : res) {
        if (!(ans.empty() && x == 0))  // 跳過前導0
            ans += (char)(x + '0');
    }
    return ans.empty() ? "0" : ans;
}
//除法
bool smaller(string a, string b) {
    if (a.size() != b.size()) return a.size() < b.size
        ();
    return a < b;
}
string div(string num1, string num2) {
    string cur = "", ans = "";
    for (char c : num1) {
        cur += c;
        // 去掉前導0
        while (cur.size() > 1 && cur[0] == '0') cur.
            erase(cur.begin());

        int x = 0;
        while (!smaller(cur, num2)) {
            cur = sub(cur, num2);
            x++;
        }
        ans += (char)(x + '0');
    }
    // 去掉前導0
    while (ans.size() > 1 && ans[0] == '0') ans.erase(
        ans.begin());
    return ans;
}
string mod(string num1, string num2) {//餘數
    string cur = "";

    for (char c : num1) {
        cur += c;
        while (cur.size() > 1 && cur[0] == '0') cur.
            erase(cur.begin());
        while (!smaller(cur, num2)) {
            cur = sub(cur, num2);
        }
    }
    return cur;
}
```

2.4 極座標排序

```
//由+x軸往逆時針排序
struct point {
    int x, y;
};
long long cross(const point &a, const point &b) {
    return (long long) a.x * b.y - (long long) a.y * b.
        x;
}
bool cmp(const point &a, const point &b) {
    int ah = (a.y < 0 or (a.y == 0 and a.x < 0));
    int bh = (b.y < 0 or (b.y == 0 and b.x < 0));
    if (ah != bh) return ah < bh;
    return cross(a, b) > 0;
}
void argument_sort(vector<point> &points) {
    sort(points.begin(), points.end(), cmp);
}
```

2.5 ExGCD

```
//ax+by=gcd(a,b)
PII gcd(int a, int b){
if(b == 0) return {1, 0};
PII q = gcd(b, a % b);
return {q.second, q.first - q.second * (a / b)};
}
```

2.6 Phi

```
//歐拉函數 phi(n) 是指<=n中與n互質的數的個數
int _phi(int n) {  // O(sqrtN)
  int res = n, a = n;
  for (int i = 2; i * i <= a; i++) {
    if (a % i == 0) {
      res = res / i * (i - 1);
      while (a % i == 0) a /= i;
    }
  }
  if (a > 1) res = res / a * (a - 1);
  return res;
}

int phi[MXN];  // 建表 最大 1e7
void phi_table(int n) {
  phi[1] = 1;
  for (int i = 2; i <= n; ++i) {
    if (phi[i]) continue;
    for (int j = i; j <= n; j += i) {
      if (phi[j] == 0) phi[j] = j;
      phi[j] = phi[j] / i * (i - 1);
    }
  }
}
```

3 Binary Search

3.1 二分搜

```
int bsearch_1(int l, int r)
{
    while (l < r)
    {
        int mid = l + r >> 1;
        if (check(mid)) r = mid;
        else l = mid + 1;
    }
    return l;
}
// ................oooooooooo
int bsearch_2(int l, int r)
{
    while (l < r)
    {
        int mid = l + r + 1 >> 1;
        if (check(mid)) l = mid;
        else r = mid - 1;
    }
    return l;
}
// ooooooooo...................
```

```
int m = *ranges::partition_point(views::iota(0LL,(int)1
    e9+9),[&](int a){
    return check(a) > k;
    });
//[begin,last)
//1111111000000000000
//搜左邊數過來第一個 0
//都是 1 會回傳 last
int partitionpoint(int L,int R,function<bool(int)> chk)
    {
  int l = L,r = R-1;
  while(r - l > 10){
    int m = l + (r-l)/2;
    if(chk(m)) l = m;
    else r = m;
  }
    int m = l;
    while(m <= r){
        if(!chk(m)) break;
        ++m;
    }
  if(!chk(m)) return m;
  else return R;
}
//手工
```

3.2 三分搜

```
int l = 1,r = 100;
while(l < r) {
    int lmid = l + (r - l) / 3; // l + 1/3區間大小
    int rmid = r - (r - l) / 3;  // r - 1/3區間大小
    lans = cal(lmid),rans = cal(rmid);
    // 求凹函數極小直
    if(lans <= rans) r = rmid - 1;
    else l = lmid + 1;
}
```

4 DataStructure

4.1 DSU(並查集)

```
struct DSU {
    vector<int> f, sz;
    int cnt;
    DSU(int n) : f(n), sz(n) {
        for (int i = 0; i < n; i++) {
            f[i] = i;
            sz[i] = 1;
        }
    }
    int find(int x) {
        if (x == f[x]) return x;
        return f[x] = find(f[x]);
    }
    void merge(int x, int y) {
        x = find(x); y = find(y);
        if (x == y) return;
        if (sz[x] < sz[y])
            swap(x, y);
        f[y] = x;
        sz[x] += sz[y];
        cnt--;
    }
    bool same(int a, int b) {
        return find(a) == find(b);
    }
    int groups() {//找有幾群
        return cnt;
    }
};
```

4.2 FenwickTree/BIT

```
struct Fenwick {//這個是1base的
    int n;
    vector<int> s;
    int lowbit(int x) { return x & -x; }
    Fenwick(int _n) {
        n = _n + 1;
        s.clear(), s.resize(n, 0);
    }
    void update(int i, int v) {  // 更新數值
        for (; i < n; i += lowbit(i))
            s[i] += v;
    }
    int query(int x) {  // 查詢前綴和(有包含)
        int pre = 0;
        for (; x; x -= lowbit(x))
            pre += s[x];
        return pre;
    }
    Fenwick(vector<int> a) {// 建構資料結構
        n = a.size(); // a 是 0-based
        s.clear(), s.resize(n + 1, 0); // s 是 1-based
        for (int i = 1; i <= n; i++)
            update(i, a[i - 1]);
    }
};
//使用:
Fenwick fw(a);//a is vector
Fenwick fw(n);//n is int
```

4.3 二維 BIT

```
struct BIT {
  int n;
  vector<vector<ll>> bit;
  int lowbit(int x) { return x & -x; }
  BIT(int _n) : n(_n + 1) {
    bit.assign(n, vector<ll>(n, 0));
  }
  void update(int x, int y, ll val) {  // 更新 (x, y)
    for (int i = x; i < n; i += lowbit(i))
      for (int j = y; j < n; j += lowbit(j))
        bit[i][j] += val;
  }
  ll query(int x, int y) {  // 查詢 (1, 1) ~ (x, y) 的
      和
    ll ans = 0;
    for (int i = x; i; i -= lowbit(i))
      for (int j = y; j; j -= lowbit(j)) ans += bit[i][
          j];
    return ans;
  }
  ll range_query(int a, int b, int x, int y) {
    return query(x, y) - query(x, b - 1) -
           query(a - 1, y) + query(a - 1, b - 1);
  }
};
```

4.4 Seg/線段樹

```
struct Seg {
#define cl(x) (x << 1)
#define cr(x) (x << 1) + 1
    int _n;
    vector<int> f, tag;
    vector<int> arr;
    //Seg(int n) : _n(n), f(4 * n), tag(4 * n) {}
    Seg(vector<int> a) {
        _n = a.size();
        f.resize(4 * _n + 1, 0);
        tag.resize(4 * _n + 1, 0);
        arr.resize(_n);
        for (int i = 0; i < _n; i++) {
            arr[i] = a[i];
        }

        build(1, 0, _n - 1);
    }
    Seg(int x){
        _n = x;
        f.resize(4 * _n + 1, 0);
        tag.resize(4 * _n + 1, 0);
        arr.resize(_n);

    }

    void build(int id, int l, int r) {
        if (l == r) {
            f[id] = arr[l];
```

```
            return;
        }
        int mid = (l + r) >> 1;
        build(cl(id), l, mid);
        build(cr(id), mid + 1, r);
        pull(id, l, r);
    }

    void pull(int id, int l, int r) {
        int mid = (l + r) >> 1;
        push(cl(id), l, mid);
        push(cr(id), mid + 1, r);
        f[id] = f[cl(id)] + f[cr(id)];
    }

    void push(int id, int l, int r) {
        if (tag[id]) {
            f[id] += tag[id] * (r - l + 1);
            if (l != r) {
                tag[cl(id)] += tag[id];
                tag[cr(id)] += tag[id];
            }
            tag[id] = 0;
        }
    }

    void update(int id, int l, int r, int x, int v) {
        if (l == r) {
            f[id] = v;
            return;
        }
        int mid = (l + r) >> 1;
        if (x <= mid) update(cl(id), l, mid, x, v);
        if (mid < x)  update(cr(id), mid + 1, r, x, v);
        pull(id, l, r);
    }

    void update(int id, int l, int r, int ql, int qr,
     int v) {
        push(id, l, r);
        if (ql <= l and qr >= r) {
            tag[id] += v;
            return;
        }
        int mid = (l + r) >> 1;
        if (ql <= mid) {
            update(cl(id), l, mid, ql, qr, v);
        }
        if (qr > mid) {
            update(cr(id), mid + 1, r, ql, qr, v);
        }
        pull(id, l, r);
    }

    int query(int id, int l, int r, int sl, int sr) {
        push(id, l, r);
        if (sl <= l and sr >= r) {
            return f[id];
        }
        int res = 0;
        int mid = (l + r) >> 1;
        int ll = 0;
        int rr = 0;
        if (sl <= mid) {
            ll = query(cl(id), l, mid, sl, sr);
        }
        if (sr > mid) {
            rr = query(cr(id), mid + 1, r, sl, sr);
        }
        res = ll + rr;
        return res;
    }
};
```

5 Graph

5.1 Topo/拓樸

```
vector<int> edge[MAXN], ans;
int deg[MAXN];

void topo(int n) {
    queue<int> q;

    for (int i = 1; i <= n; i++)
        if (!deg[i])
            q.push(i);

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        ans.push_back(u);

        for (int v : edge[u]) {
            deg[v]--;
            if (!deg[v])
                q.push(v);
        }
    }
}
```

5.2 bellman/邊長鬆弛

```
struct Edge {
  int from, to, weight;
} edge[MAXN];
int dis[MAXN];
void bellmen_ford(int n, int m) {
  dis[1] = 0;  // 起點的距離一定是 0
  for (int j = 0; j < n - 1;
       j++) {  // n 個節點要跑 n - 1 次
    for (int i = 0; i < m; i++) {  // 對於所有邊都嘗試
        鬆弛
      if (dis[edge[i].to] >
          dis[edge[i].from] + edge[i].weight)
        dis[edge[i].to] =
            dis[edge[i].from] + edge[i].weight;
    }
  }
}
```

5.3 ola/歐拉

```
//歐拉路徑(無向圖)兩個奇數點或者全偶點
//歐拉路徑(有向圖)條件:起點out=in+1,終點in=out+1,其他點
    in=out
//歐拉迴路(無向圖)所有點偶數點
//歐拉迴路(有向圖)所有點in=out
vector<int> anse;//邊
vector<int> ansp;//點
vector<bool> vis(m,false);
function<void(int)> dfs=[&](int u){
    while(!mp[u].empty()){
        auto [x,e]=mp[u].back();
        mp[u].pop_back();
        if(vis[e]){
            continue;
        }
        vis[e]=true;
        dfs(x);
        anse.pb(e);
    }
    ansp.pb(u);
};
```

5.4 LCA/最近公共祖先

```
struct{
    int n, logN;
    vector<vector<int>>jump;
    vector<int>in, out;
    bool is_ancestor(int x, int y){
        if(x == -1 || y == -1) return 1;
        return in[x] <= in[y] && out[y] <= out[x];
    }
    int query(int x, int y){
        if(is_ancestor(x, y)) return x;
        if(is_ancestor(y, x)) return y;
        for(int i = logN - 1; i >= 0; i--){
            if(!is_ancestor(jump[x][i], y)) x = jump[x
                ][i];
        }
        return jump[x][0];
```

```
    }
    void init(int _n, vector<vector<int>>&v){//size,
        adj array 0base
        n = _n;
        logN = log2(n) + 1;

        jump.resize(n, vector<int>(logN, -1));
        in.resize(n); out.resize(n);

        int timing = 1;
        auto dfs = [&](auto &&self, int x) -> void{
            in[x] = timing++;
            for(auto &i : v[x]){
                jump[i][0] = x;
                self(self, i);
            }
            out[x] = timing++;
            return;
        }; dfs(dfs, 0);

        for(int i = 1; i < logN; i++){
            for(int now = 1; now < n; now++){
                if(jump[now][i-1] == -1) continue;
                jump[now][i] = jump[jump[now][i-1]][i
                    -1];
            }
        }
    }
}LCA;
```

5.5 ConvexHull/凸包

```
struct node{
    int x,y;
    bool operator<(const node &other) const{
        return (x<other.x) || (x==other.x && y<other.y)
            ;
    };
};

struct ConvexHull{
    vector<node> vec;
    int n;
    void init(vector<node> &v){
        vec=v;
        n=v.size();
    }

    int cross(node a,node o,node b){
        return (a.x-o.x)*(b.y-o.y)-(b.x-o.x)*(a.y-o.y);
    }

    vector<int> point;
    vector<int> area;

    void build(){
        point=vector<int>(n+1);
        area=vector<int>(n+1);
        vector<node> up,down;
        int upsum=0;
        int downsum=0;
        for(int i=0;i<n;i++){
            while(up.size()>=2 && cross(up[up.size()
                -2],up[up.size()-1],vec[i])<=0){
                upsum-=cross(up[up.size()-2],{0,0},up[
                    up.size()-1]);
                up.pop_back();

            }
            up.push_back(vec[i]);
            if(up.size()>1) upsum+=cross(up[up.size()
                -2],{0,0},up[up.size()-1]);

            while(down.size()>=2 && cross(down[down.
                size()-2],down[down.size()-1],vec[i])
                >=0){
                downsum-=cross(down[down.size()
                    -2],{0,0},down[down.size()-1]);
                down.pop_back();
            }
            down.push_back(vec[i]);
            if(down.size()>1) downsum+=cross(down[down.
                size()-2],{0,0},down[down.size()-1]);

            area[i+1]=abs(upsum-downsum);

            if(i+1==1) point[i+1]=1;
            else point[i+1]=up.size()+down.size()-2;
        }
    }

    double getarea(int p){
        return area[p]/2.0;
    }

    int getnum(int p){
        return point[p];
    }
};
```

5.6 km/二分圖最大權匹配

```
struct KM{ // max weight, for min negate the weights
  int n, mx[MXN], my[MXN], pa[MXN];
  ll g[MXN][MXN], lx[MXN], ly[MXN], sy[MXN];
  bool vx[MXN], vy[MXN];
  void init(int _n) { // 1-based, N個節點
    n = _n;
    for(int i=1; i<=n; i++) fill(g[i], g[i]+n+1, 0);
  }
  void addEdge(int x, int y, ll w) {g[x][y] = w;} //左
      邊的集合節點x連邊右邊集合節點y權重為w
  void augment(int y) {
    for(int x, z; y; y = z)
      x=pa[y], z=mx[x], my[y]=x, mx[x]=y;
  }
  void bfs(int st) {
    for(int i=1; i<=n; ++i) sy[i]=INF, vx[i]=vy[i]=0;
    queue<int> q; q.push(st);
    for(;;) {
      while(q.size()) {
        int x=q.front(); q.pop(); vx[x]=1;
        for(int y=1; y<=n; ++y) if(!vy[y]){
          ll t = lx[x]+ly[y]-g[x][y];
          if(t==0){
            pa[y]=x;
            if(!my[y]){augment(y);return;}
            vy[y]=1, q.push(my[y]);
          }else if(sy[y]>t) pa[y]=x,sy[y]=t;
      } }
      ll cut = INF;
      for(int y=1; y<=n; ++y)
        if(!vy[y]&&cut>sy[y]) cut=sy[y];
      for(int j=1; j<=n; ++j){
        if(vx[j]) lx[j] -= cut;
        if(vy[j]) ly[j] += cut;
        else sy[j] -= cut;
      }
      for(int y=1; y<=n; ++y) if(!vy[y]&&sy[y]==0){
        if(!my[y]){augment(y);return;}
        vy[y]=1, q.push(my[y]);
  } } }
  ll solve(){ // 回傳值為完美匹配下的最大總權重
    fill(mx, mx+n+1, 0); fill(my, my+n+1, 0);
    fill(ly, ly+n+1, 0); fill(lx, lx+n+1, -INF);
    for(int x=1; x<=n; ++x) for(int y=1; y<=n; ++y) //
        1-base
      lx[x] = max(lx[x], g[x][y]);
    for(int x=1; x<=n; ++x) bfs(x);
    ll ans = 0;
    for(int y=1; y<=n; ++y) ans += g[my[y]][y];
    return ans;
} }graph;
```

6 dynamic programming

6.1 SOS DP

```
// 求子集和 或超集和 -> !(mask & (1 << i))
for(int i = 0; i<(1<<N); ++i) F[i] = A[i]; //預處理 狀
    態權重
```

```
for(int i = 0;i < N; ++i)
for (int s = 0; s < (1<<N) ; ++s)
  if (s & (1 << i))
    F[s] += F[s ^ (1 << i)];

//窮舉子集合
for(int s = mask; s ; s = (s-1)&mask;)
//放這個做啥我不會用QAQ
```

7 Flow

7.1 Dinic

```
struct Dinic{
  struct Edge{ int v,f,re; };
  int n,s,t,level[MXN];
  vector<Edge> E[MXN];
  void init(int _n, int _s, int _t){
    n = _n; s = _s; t = _t;
    for (int i=0; i<n; i++) E[i].clear();
  }
  void add_edge(int u, int v, int f){
    E[u].PB({v,f,SZ(E[v])});
    E[v].PB({u,0,SZ(E[u])-1});
  }
  bool BFS(){
    for (int i=0; i<n; i++) level[i] = -1;
    queue<int> que;
    que.push(s);
    level[s] = 0;
    while (!que.empty()){
      int u = que.front(); que.pop();
      for (auto it : E[u]){
        if (it.f > 0 && level[it.v] == -1){
          level[it.v] = level[u]+1;
          que.push(it.v);
    } } }
    return level[t] != -1;
  }
  int DFS(int u, int nf){
    if (u == t) return nf;
    int res = 0;
    for (auto &it : E[u]){
      if (it.f > 0 && level[it.v] == level[u]+1){
        int tf = DFS(it.v, min(nf,it.f));
        res += tf; nf -= tf; it.f -= tf;
        E[it.v][it.re].f += tf;
        if (nf == 0) return res;
    } }
    if (!res) level[u] = -1;
    return res;
  }
  int flow(int res=0){
    while ( BFS() )
      res += DFS(s,2147483647);
    return res;
} }flow;
/*flow.init(n, source, sink); : 初始化點數、源點、匯點
flow.add_edge(u, v, w); : 加入一條點u連到點v容量為w的邊
flow.flow(); : 回傳值為最大流量
*/
```

8 String

8.1 SAIS

```
// 此為後綴陣列模板
// 用法:
// 把要處理的字串轉成整數陣列傳入suffix_array()，會得到
    SA與H雛結果
const int N = 500010;  // 有可能要在這就寫原本字串的兩
    倍長
struct SA {
#define REP(i, n) for (int i = 0; i < int(n); i++)
#define REP1(i, a, b) for (int i = (a); i <= int(b); i
    ++)
  bool _t[N * 2];
  int _s[N * 2], _sa[N * 2], _c[N * 2], x[N], _p[N],
      _q[N * 2], hei[N], r[N];
  int operator[](int i) { return _sa[i]; }
  void build(int* s, int n, int m) {
    memcpy(_s, s, sizeof(int) * n);
    sais(_s, _sa, _p, _q, _t, _c, n, m);
    mkhei(n);
  }
  void mkhei(int n) {
    REP(i, n) r[_sa[i]] = i;
    hei[0] = 0;
    REP(i, n) if (r[i]) {
      int ans = i > 0 ? max(hei[r[i - 1]] - 1, 0) : 0;
      while (_s[i + ans] == _s[_sa[r[i] - 1] + ans])
        ans++;
      hei[r[i]] = ans;
    }
  }
  void sais(int* s, int* sa, int* p, int* q, bool* t,
        int* c, int n, int z) {
    bool uniq = t[n - 1] = true, neq;
    int nn = 0, nmxz = -1, *nsa = sa + n, *ns = s + n,
        lst = -1;
#define MS0(x, n) memset((x), 0, n * sizeof(*(x)))
#define MAGIC(XD)                                  \
  MS0(sa, n);                                      \
  memcpy(x, c, sizeof(int) * z);                   \
  XD;                                              \
  memcpy(x + 1, c, sizeof(int) * (z - 1));         \
  REP(i, n)                                        \
  if (sa[i] && !t[sa[i] - 1])                      \
    sa[x[s[sa[i] - 1]]++] = sa[i] - 1;             \
  memcpy(x, c, sizeof(int) * z);                   \
  for (int i = n - 1; i >= 0; i--)                 \
    if (sa[i] && t[sa[i] - 1])                     \
      sa[--x[s[sa[i] - 1]]] = sa[i] - 1;
    MS0(c, z);
    REP(i, n) uniq &= ++c[s[i]] < 2;
    REP(i, z - 1) c[i + 1] += c[i];
    if (uniq) {
      REP(i, n) sa[--c[s[i]]] = i;
      return;
    }
    for (int i = n - 2; i >= 0; i--)
      t[i] =
          (s[i] == s[i + 1] ? t[i + 1] : s[i] < s[i +
              1]);
    MAGIC(REP1(i, 1, n - 1) if (t[i] && !t[i - 1])
              sa[--x[s[i]]] = p[q[i] = nn++] = i);
    REP(i, n) if (sa[i] && t[sa[i]] && !t[sa[i] - 1]) {
      neq = lst < 0 || memcmp(s + sa[i], s + lst,
                              (p[q[sa[i]] + 1] - sa[i])
                                  *
                              sizeof(int));
      ns[q[lst = sa[i]]] = nmxz += neq;
    }
    sais(ns, nsa, p + nn, q + n, t + n, c + z, nn,
         nmxz + 1);
    MAGIC(for (int i = nn - 1; i >= 0; i--)
              sa[--x[s[p[nsa[i]]]]] = p[nsa[i]]);
  }
} sa;
int H[N], SA[N];  // SA: 後綴陣列的length
                  // H[i]: SA[i]與SA[i -1]的共同子字串
                     長
void suffix_array(int* ip, int len) {
  // should padding a zero in the back
  // ip is int array, len is array length
  // ip[0..n-1] != 0, and ip[len] = 0
  ip[len++] = 0;
  sa.build(ip, len, 128);
  for (int i = 0; i < len; i++) {
    H[i] = sa.hei[i + 1];
    SA[i] = sa._sa[i + 1];
  }
  // resulting height, sa array \in [0,len)
}
```

9 酷東西可能會用到

9.1 areaofrect/矩形聯集面積

```
struct AreaofRectangles{
#define cl(x) (x<<1)
#define cr(x) (x<<1|1)
    int n, id, sid;
```

```cpp
    pair<int,int> tree[MXN<<3];   // count, area
    vector<int> ind;
    tuple<int,int,int,int> scan[MXN<<1];
    void puint(int i, int l, int r){
        if(tree[i].first)  tree[i].second = ind[r+1] - ind[l];//當前區間有被做加值
        else if(l != r){//沒有就直接拿兩個子節點的第二個數字
            int mid = (l+r)>>1;
            tree[i].second = tree[cl(i)].second + tree[cr(i)].second;
        }
        else    tree[i].second = 0;
    }
    void upd(int i, int l, int r, int ql, int qr, int v){
        if(ql <= l && r <= qr){
            tree[i].first += v;
            puint(i, l, r); return;
        }
        int mid = (l+r) >> 1;
        if(ql <= mid) upd(cl(i), l, mid, ql, qr, v);
        if(qr > mid) upd(cr(i), mid+1, r, ql, qr, v);
        puint(i, l, r);
    }
    void init(int _n){
        n = _n; id = sid = 0;
        ind.clear(); ind.resize(n<<1);
        fill(tree, tree+(n<<2), make_pair(0, 0));
    }
    void addRectangle(int lx, int ly, int rx, int ry){
        ind[id++] = lx; ind[id++] = rx;
        scan[sid++] = make_tuple(ly, 1, lx, rx);
        scan[sid++] = make_tuple(ry, -1, lx, rx);
    }
    int solve(){
        sort(ind.begin(), ind.end());
        ind.resize(unique(ind.begin(), ind.end()) - ind.begin());//離散化
        sort(scan, scan + sid);
        int area = 0, pre = get<0>(scan[0]);
        for(int i = 0; i < sid; i++){
            auto [x, v, l, r] = scan[i];
            area += tree[1].second * (x-pre);//
            upd(1, 0, ind.size()-1, lower_bound(ind.begin(), ind.end(), l)-ind.begin(),
                lower_bound(ind.begin(),ind.end(),r)-ind.begin()-1, v);
            pre = x;
        }
        return area;
    }
} }rect;
```

10 Math2

10.1 Theorem

- Lucas's Theorem :
  For $n, m \in \mathbb{Z}^*$ and prime P, $C(m, n)$ mod $P = \Pi(C(m_i, n_i))$ where m_i is the i-th digit of m in base P.

- Stirling approximation :
  $n! \approx \sqrt{2\pi n}(\frac{n}{e})^n e^{\frac{1}{12n}}$

- Stirling Numbers(permutation $|P| = n$ with k cycles):
  $S(n, k) =$ coefficient of x^k in $\Pi_{i=0}^{n-1}(x + i)$

- Stirling Numbers(Partition n elements into k non-empty set):
  $S(n, k) = \frac{1}{k!}\sum\limits_{j=0}^{k}(-1)^{k-j}\binom{k}{j}j^n$

- Pick's Theorem : $A = i + b/2 - 1$
  A: Area、i: grid number in the inner、b: grid number on the side

- Catalan number : $C_n = \binom{2n}{n}/(n + 1)$
  $C_n^{n+m} - C_{n+1}^{n+m} = (m + n)!\frac{n-m+1}{n+1} \quad for \quad n \geq m$
  $C_n = \frac{1}{n+1}\binom{2n}{n} = \frac{(2n)!}{(n+1)!n!}$
  $C_0 = 1 \quad and \quad C_{n+1} = 2(\frac{2n+1}{n+2})C_n$
  $C_0 = 1 \quad and \quad C_{n+1} = \sum_{i=0}^{n} C_i C_{n-i} \quad for \quad n \geq 0$

- Euler Characteristic:
  planar graph: $V - E + F - C = 1$
  convex polyhedron: $V - E + F = 2$
  V, E, F, C: number of vertices, edges, faces(regions), and components

- Kirchhoff's theorem :
  $A_{ii} = deg(i), A_{ij} = (i, j) \in E \ ? -1 : 0$, Deleting any one row, one column, and cal the det(A)

- Polya' theorem (c is number of color，m is the number of cycle size):
  $(\sum_{i=1}^{m} c^{gcd(i,m)})/m$

- Burnside lemma:
  $|X/G| = \frac{1}{|G|}\sum\limits_{g \in G}|X^g|$

- 錯排公式: (n 個人中，每個人皆不再原來位置的組合數):
  $dp[0] = 1; dp[1] = 0;$
  $dp[i] = (i - 1) * (dp[i - 1] + dp[i - 2]);$

- Bell 數 (有 n 個人, 把他們拆組的方法總數) :
  $B_0 = 1$
  $B_n = \sum_{k=0}^{n} s(n, k) \quad (second - stirling)$
  $B_{n+1} = \sum_{k=0}^{n}\binom{n}{k}B_k$

- Wilson's theorem :
  $(p - 1)! \equiv -1(mod\ p)$

- Fermat's little theorem :
  $a^p \equiv a(mod\ p)$

- Euler's totient function:
  $A^{B^C} mod\ p = pow(A, pow(B, C, p - 1))mod\ p$

- 歐拉函數降冪公式:
  $A^B \text{ mod } C = A^{B \text{ mod } \phi(c)+\phi(c)} \text{ mod } C$

- 6 的倍數:
  $(a - 1)^3 + (a + 1)^3 + (-a)^3 + (-a)^3 = 6a$

- Standard young tableau (標準楊表):
  $\lambda = (\lambda_1 \geq \cdots \geq \lambda_k), \sum \lambda_i = n$ denoted by $\lambda \vdash n$
  $\lambda \vdash n$ 意思為 λ 整數拆分 n eg. $n = 10, \lambda = (6, 4)$ 此拆分可表示一種楊表形狀。
  楊表: 第 1 列 λ_1 行 $\cdots$ 第 k 列 λ_k 行的方格圖。
  標準楊表: 每列從左到右遞增，每行從上到下遞增。
  Let T 為某一 Permutation 跑 RSK 後的標準楊表，則此 Permutation 的 LDS、LIS 長度分別為 T 的列、行數。

- RSK Correspondence:
  A permutation is bijective to (P, Q) 一對標準楊表
  P : Permutation 跑 RSK 算法的結果，可為半標準楊表。
  Q : 可用來還原 Permutation (像排列矩陣)。

- Hook length formula (形狀為 λ 的標準楊表個數):
  $f^\lambda = \frac{n!}{\prod h_\lambda(i,j)}$
  $h_\lambda(i, j) =$ number of pair (x, y) where $(x = i \vee y = j) \wedge (x, y) \geq (i, j)$ 且 (x, y) 落在形狀為 λ 的表上。
  Recursion:
  (i) $f^{(0,\cdots,0)} = 1$
  (ii) $f^{(\lambda_1,\cdots,\lambda_m)} = \sum_{k=1}^{m} f^{(\lambda_1,\cdots,\lambda_{k-1},\lambda_k-1,\lambda_{k+1},\cdots,\lambda_m)}$

- 一個環上相鄰顏色不同的染色數量:
  $f(n, k) = (k - 1)^n + (-1)^n(k - 1)$
  n 是點的數量，k 是顏色的數量

10.2 Prime

```
/* 12721, 13331, 14341, 75577, 123457, 222557, 556679
* 999983, 1097774749, 1076767633, 100102021, 999997771 *
1001010013, 1000512343, 987654361, 999991231 * 999888733,
98789101, 987777733, 999991921, 1010101333 * 1010102101,
1000000000039, 10000000000000037 * 2305843009213693951,
4611686018427387847 * 9223372036854775783, 18446744073709551557 */
```